

La consulta de caja, de punta a punta

Reportes · Seccion de caja · Frontend + Backend + Supabase

Que se consulta, en que orden, por que el filtro va al servidor, y que datos ya venían disponibles sin pedir nada nuevo.

0. El mapa completo en una pagina

Una caja con 12 turnos = 1 + 1 + 12 = 14 SELECT, todos en una transaction

NAVEGADOR (Angular)SPRING BOOTSUPABASE

```
onBoxSelected(boxId)
  %% ReportService.getCashBoxReport()
  %% GET /api/local/reports/cash-box/{boxId}
  ?from=&to=&userId= ReportController:148
  %%
  %% > ReportsImpl.getCashBoxReport()
  %% CashBoxRepository.findById(boxId)
  %% CashRegisterRepository
  %% .findByCashBoxIdOrderByOpenedAt
  %% por cada turno:
  SaleRepository.findByCashRegister(turno)
  %%
  %% > filtro de usuario en JAVA
  filtrado= userId!=null||from||to

<- CashBoxReportDTO { boxId, number, sessions[], totalSessions }

next: (data) => {
  this.cashBoxReport = data; // la tabla
  calcularCorteDeCaja(data); // el corte: SUMA de las filas
  if(!reporteFiltrado) recalcularUsuariosDeLaCaja();
}

NO hay una segunda consulta para el corte.
```

Ese es el punto central de este documento, y conviene verlo de entrada: el corte de caja que se pidio NO es una consulta. Es una suma de las filas que la tabla ya tiene en la pantalla. Por eso no se documento un endpoint nuevo y no hay nada que pueda quedar desfasado de lo que el usuario ve.

CAPA	QUE APORTA
cash_boxes	El inventario: CAJA 1, CAJA 2... Cambia poco.
cash_registers	Los TURNOS: una fila por apertura. Es donde vive el dinero.

CAPA	QUE APORTA
sales	Las ventas, cada una con su turno y su usuario.
sale_details	El costo de cada articulo: sirve para la utilidad.
users	Quien vendio. Null en ventas anteriores a V5.

1. Las tres tablas y por que el filtro va al servidor

1.1 Caja, turno y venta: tres cosas distintas

El cambio de V4 a V5 fue el que ordeno todo esto, y es importante tener claro por que las tres tablas no son la misma:

TABLA	QUE ES	CRECE CON
cash_boxes	La caja fisica del local	Lentamente: 2 o 3 en un local
cash_registers	El TURNO: una apertura y su cierre	Un renglon por dia de trabajo
sales	Las ventas de un turno	Un renglon por ticket

Una caja abierta diez veces produce DIEZ filas en `cash\_registers`, todas con el mismo numero "CAJA 1". Por eso el selector de Reportes lista cajas y no turnos: "CAJA 1" es lo que el cajero reconoce, y pedirle que elija entre "CAJA 1" y "CAJA 1" dos veces es pedirle que adivine.

1.2 Por que el filtro de usuario NO va en el navegador

Esta decision ya esta tomada y documentada en el codigo, pero vale la pena repetirla porque es la que sostiene medio modulo:

El filtro viaja al backend

```
// reports.ts, loadCashBoxReport()
this.reportService.getCashBoxReport(boxId, {
  from: this.from,
  to: this.to,
  userId: this.cashBoxUserFilter,
}).subscribe(...);

// El filtro va en la URL, no en el .filter() del template.
```

- Si se filtrara en el navegador habria que traer las ventas de TODOS los turnos para descartar la mayoria.
- Los totales de la tabla no coincidirian con los del backend, y el usuario no tendria forma de saber cual era el correcto.
- `difference` (la diferencia del corte) es un dato CONGELADO al cerrar el turno. Al lado de un total filtrado daria un descuadre que no ocurrio.

LA DIFERENCIA ES LO MAS IMPORTANTE DE ESTA SECCION

No es un numero calculado: es lo que el cajero conto al cerrar. Cuando hay filtro, el backend lo manda en `null` y la tabla lo OCULTA en vez de pintar un cero. Por eso la columna tiene `\*ngIf="!s.filtrado"` y no un `number` pipe cualquiera. El corte nuevo respeta la misma regla y pone un "—", con una nota que explica por que.

1.3 El filtro de fecha se aplica sobre openedAt, no sobre closedAt

El filtro de fechas, en el service

```
// ReportsImpl.java:373-377
LocalDate start = from;
LocalDate fin = to == null ? null : to.plusDays(1);

for(CashRegisterEntity sesion : sesiones){
    LocalDate dia = sesion.getOpenedAt().toLocalDate();
    if(start != null && dia.isBefore(start)) continue;
    if(fin != null && !dia.isBefore(fin)) continue;
}
```

Un turno abierto a las 11pm y cerrado a las 2am cae en el dia en que se ABRIO. Es lo que el cajero recuerda: el turno es de la noche del viernes, no del sabado. El `to.plusDays(1)` convierte un "hasta" inclusivo en un limite exclusivo, para no perder el ultimo dia.

EL FILTRO CORRE EN JAVA, Y ES A PROPOSITO

Una caja tiene pocos turnos, asi que el conjunto es chico y se filtra en memoria sin penalizar. A cambio, UNA sola pasada de codigo calcula ventas, agrupado por vendedor, utilidad y diferencia. Hacerlo en SQL serian cuatro consultas distintas y despues reconciliarlas en Java, que es justo donde se esconden los bugs de redondeo y de descuadre.

2. Los tres bugs que havia, y por que pasaban

Los tres se reportaron como "la seccion de caja no se usa bien". Tenian tres causas distintas, en tres capas distintas, y ninguno estaba donde se miraba.

2.1 La tabla desaparecia al filtrar por usuario

La causa: un null en el lugar equivocado

```
// reports.ts (ANTES)
applyCashBoxUserFilter() {
  ...
  this.cashBoxReport = null; // borra TODO
  this.loadCashBoxReport(this.selectedBoxId);
}

// reportes.html:202
<div class="table-container" *ngIf="cashBoxReport"> // se lleva la seccion entera
```

No era un parpadeo. El `\*ngIf` envolvía la tabla, el desplegable de usuario y el título de la caja, todos juntos. Durante cada petición no había nada en pantalla, y como el control para volver a "Todos" estaba dentro del mismo bloque, tampoco había forma de volver.

QUE	ANTES	AHORA
Al cambiar de USUARIO	La tabla se borraba	Se queda, atenuada, con "Actualizando..."
Al cambiar de CAJA	La tabla se borra	Igual: el título cambiaría y ver CAJA 1 bajo CAJA 2 es peor
Como se avisa	Nada	boxLoading + opacity 0.45 + pointer-events: none

POR QUE EL CONTRASTE ES INTENCIONAL

Los dos casos se ven iguales en el código pero NO son lo mismo. Al cambiar de usuario la caja no cambia: el título sigue diciendo CAJA 3 y las filas son las mismas, solo que recortadas. Borrarlas no evita ninguna confusión. Al cambiar de caja, ver las filas de CAJA 1 bajo el título de CAJA 2 SI es una confusión real, y por ahí sí se limpia.

El `pointer-events: none` no es decorativo. Sin él, el usuario puede clicar una fila mientras sus datos están siendo reemplazados, y el detalle que se abriera sería el de la fila anterior.

2.2 Los otros usuarios desaparecían del desplegable

Este sí es un bug de datos, y la causa estaba en el backend:

El backend descarta los turnos que no tienen ventas del usuario

```
// ReportsImpl.java:399-411
List<SaleEntity> filtradas = userId == null
    ? ventas
    : ventas.stream().filter(v -> userId.equals(v.getUser().getId())).toList();

if(filtrado && filtradas.isEmpty()){
  continue; // el turno desaparece COMPLETO
}
```

Y el frontend armaba el menú con lo que llegó

```
// reports.ts (ANTES)
private recalcularUsuariosDeLaCaja() {
  for (const sesion of this.cashBoxReport.sessions) {
    for (const v of sesion.sellers) {
      porId.set(v.userId, ...); // -armed con lo que LLEGO filtrado
    }
  }
  this.boxReportUsers = [...porId.values()].sort(...);
}
```

PASO	QUE PASABA
1	El usuario elige "Juan" en el desplegable
2	El backend devuelve SOLO los turnos con ventas de Juan
3	El frontend rearma el menu con los vendedores de esos turnos
4	El menu queda con "Juan" y nada mas
5	Para recuperar a los otros hay que volver a "Todos"

Un menu que se come a si mismo. La correccion no fue cambiar el backend (su comportamiento es correcto: con filtro, un turno sin ventas del usuario no debe aparecer), sino dejar de reconstruir la lista cuando la respuesta viene recortada:

La correccion: un getter, una condicion

```
// reports.ts (AHORA)
private get reporteFiltrado(): boolean {
  return this.usuarioAplicado !== null || !!this.from || !!this.to;
}

next: (data) => {
  this.cashBoxReport = data;
  calcularCorteDeCaja(data);
  if(!this.reporteFiltrado) { // solo con respuesta COMPLETA
    this.recalcularUsuariosDeLaCaja();
  }
}
```

EL GETTER DEDUCE DE LO QUE SE MANDO, NO DE LO QUE LLEGO

Intentar deducirlo de las sesiones NO funciona: si el filtro no encuentra nada, `sessions` viene vacio y no hay forma de distinguir "esta caja no tiene turnos" de "el filtro no coincidio con ninguno". La unica fuente confiable es lo que se pidio.

2.3 El corte de caja mostraba el turno equivocado

Este es el mas grave de los tres, porque no era una molestia: era un numero incorrecto.

El endpoint que usaba el corte viejo

```
// Reportes pedia:
this.cashService.getSummary()

// CashRegisterController.java:52 -> CashRegisterImpl.java:665
public CashSummaryResponse getSummary(){
    CashRegisterEntity cash = repository.findByActiveTrue()
        .orElseThrow(() -> new CashException("No existe caja abierta"));
    return calculateSummary(cash);
}
```

`findByActiveTrue()` devuelve EL TURNO ABIERTO EN ESTE MOMENTO. No sabe nada de la caja seleccionada, ni del usuario, ni de las fechas. Es una consulta correcta para su proposito (el modal de cierre de cobro, que necesita el turno abierto) y el error fue usarla en Reportes.

CON LA CAJA FILTRADA ARRIBA

CAJA 3 / Juan / octubre

LO QUE DECIA EL CORTE DE ABAJO

El turno que este abierto ahora, sea cual sea

CAJA 1 / todos / sin fechas

El turno que este abierto ahora, sea cual sea

POR QUE NO ERA "QUITAR LA SECCION"

Dos secciones bajo el mismo titulo, respondiendo preguntas distintas, con una de ellas en una pantalla de dinero: eso es peor que una seccion ausente. Un numero que no corresponde a lo que se filtra no es un bug de UI, es un bug de datos. Ademas, los botones de Imprimir / PDF / Excel leian esa misma respuesta, con lo cual el papel impreso no se podia atribuir a ninguna caja.

3. El corte nuevo: una suma, no una consulta

3.1 Los datos ya venian en la respuesta de la tabla

Antes de escribir una linea de codigo habia que preguntarse si el corte necesitaba algo que la tabla no trajera. La respuesta es no. `CashBoxSessionDTO` ya trae, POR TURNO:

Lo que YA venia en cada fila de la tabla

```
class CashBoxSessionDTO {
    openingAmount;    closingAmount;    // fondo puesto y efectivo contado
    cashSales;        debitSales;        // ventas por metodo de pago
    creditSales;      totalSales;
    expectedAmount;   // openingAmount + cashSales
    difference;       differenceReason; // congelados al cerrar
    totalTickets;     grossProfit;    // utilidad
    sellers[];        sales[];
    filtrado;          // la UI sabe si ocultar la diferencia
}
```

Sumarlas da el total de la caja. No hay endpoint nuevo, no hay una peticion extra y no hay forma de que el corte y la tabla discrepen: se calculan en el mismo turno del callback.

3.2 La regla de la diferencia

Como se decide si la diferencia es calculable

```
private calcularCorteDeCaja(data: CashBoxReportDTO) {  
  ...  
  if (x.active) { abiertos++; } else { cerrados++; }  
  
  if (x.difference !== null) {  
    diferencia += x.difference;  
  }  
  
  const sinDiferenciaUtil = this.reporteFiltrado || cerrados === 0;  
  this.corteCaja = {  
    ...  
    difference: sinDiferenciaUtil ? null : diferencia,  
  };  
}
```

La diferencia se suma SOLO sobre turnos CERRADOS, por dos razones concretas:

- Con filtro, el backend la manda en null en todos los turnos. No es cero: es que no corresponde.
- Un turno abierto todavia NO TIENE diferencia. El backend manda `sesion.getDifference()`, que se calcula al cerrar. Sumar los abiertos como cero daría un total mas chico de lo que es, sin que nada lo advierta.

POR QUE SE MUESTRA UN GUIÓN Y NO UN CERO

Un "\$0.00" afirma "el cajero no sobro ni faltó nada": es una afirmación sobre dinero real, y es falsa cuando hay un filtro. El "—" no dice nada falso, y va acompañado de una nota que explica cual de los dos motivos aplica. Un dato ausente con su razón es mas útil que un dato inventado.

3.3 Por que se movio debajo de la tabla

GRANO	PREGUNTA QUE RESPONDE	DONDE
1 fila	Como le fue a este turno	La tabla (una fila por turno)
1 total	Como le fue a esta caja	El corte, justo debajo

Son la misma información a dos granularidades, y por eso van juntas: leerlas juntas es comparar el total con el detalle. Separadas por tres pantallas, el total deja de servir como comprobación de los renglones.

3.4 El formato: por que se veia alargado

El dato concreto que hacía que la sección se sintiera inútil no era la posición ni los datos. Era el CSS: dos diseños distintos para el mismo documento.

Los dos estilos, lado a lado

```

/* cobro.css (el modal de cierre) – GRID DE 2 COLUMNAS */
.cash-summary {
  display: grid;
  grid-template-columns: 1fr 1fr;
}

/* reportes.css (el corte viejo) – COLA angosta, 9 renglones */
.corte-body {
  display: flex;
  flex-direction: column;
  max-width: 420px;           // angosto
}

```

VERSION	COLUMNAS	REGLONES	ANCHO
Modal de cobro	2	~4	Ancho del modal
Corte viejo en Reportes	1	9	420 px
Corte nuevo en Reportes	2	6	Ancho de la tarjeta

Los nueve montos apilados de a uno, en una columna de 420 pixeles. El total, que es el dato que el cajero va a buscar, quedaba debajo de todo y a veces habia que hacer scroll para verlo.

La solucion fue mover las clases al CSS global y reutilizarlas, no copiarlas:

Ahora vive UNA vez y lo usan los dos componentes

```

/* styles.css (global) */
.cash-summary { display: grid; grid-template-columns: 1fr 1fr; }
.cash-row      { display: flex; justify-content: space-between; ... }
.cash-row.total { grid-column: 1 / -1; font-weight: 700; }
.cash-row.expected { grid-column: 1 / -1; background: rgb(34,197,94,.12); ... }

```

EL CRITERIO QUE SE APLICO

Es el mismo que se uso con `.input-error`` y `.error-msg``: si dos componentes necesitan una regla, va en el global. Copiarla produce dos reglas que se desincronizan sin avisar, y el sintoma aparece meses despues como "el modal se ve distinto al reporte", que es imposible de rastrear sin saber que hubo una copia.

3.5 Las exportaciones, reapuntadas

BOTON	LEIA	AHORA LEE
Imprimir	cashSummary (/cash/summary)	corteCaja + el bloque .print-only
PDF	cashSummary (/cash/summary)	corteCaja
Excel	cashSummary (/cash/summary)	corteCaja

El detalle que hace que el papel sirva para conferir: el encabezado impreso, el PDF y el Excel usan el MISMO getter, `corteRotulo`, que arma "CAJA 3" o "CAJA 3 - Juan Perez" segun el filtro. Si cada uno escribiera su propio encabezado, el p y la pantalla podrian discrepar y no habria forma de saber cual es el bueno.

Un solo encabezado para los tres formatos

```
get corteRotulo(): string {
  const caja = this.cashBoxReport;
  if (!caja) return 'caja';
  return this.cashBoxUserFilter === null
    ? caja.number
    : `${caja.number} - ${this.nombreUsuarioFiltrado()}`;
}
```

4. Permisos: por que el corte nuevo no pide permiso propio

El endpoint viejo (`/cash/summary`) exigia `CORTE\_CAJA`. El nuevo no pide nada, porque no existe: los datos ya vinieron con `VER\_REPORTES` en la llamada del reporte de caja.

ENDPOINT	PERMISO	QUIEN LO USA
/reports/cash-box/{boxId}	VER_REPORTES	La tabla y ahora el corte
/cash/summary	CORTE_CAJA	Solo el modal de cierre de cobro
/cash/boxes	VER_CAJA	El selector de caja

UN BENEFICIO COLATERAL QUE NO SE BUSCO

Con el corte agregado, Reportes deja de necesitar el permiso CORTE_CAJA para mostrar informacion de caja. Antes un usuario con VER_REPORTES podia ver la tabla de turnos completa pero el total le salia vacio o con error de 403, segun como estuviera configurado el rol. Ahora los dos datos salen de la misma llamada y son consistentes entre si.

5. Rendimiento: cuando el filtro si va al servidor

Con 2 o 3 cajas y un turno diario, el reporte completo son 2 o 3 filas y el detalle de ventas cabe de sobra. El dia que un local tenga 50 cajas y un turno diario, el numero de SELECT crece linealmente con los turnos (uno por turno), y ahi la pregunta correcta ya no es "hay que filtrar antes" sino "hay que traer los detalles de todos los turnos cuando no se van a ver".

SITUACION	CANTIDAD DE SELECT	CONVIENE CAMBIAR
3 cajas, 3 turnos	~7	No
3 cajas, 30 turnos	~34	No
50 cajas, 1 turno cada una	~102	Tal vez: paginar los turnos

El ``sellers`` y el ``sales`` de cada turno viajan SIEMPRE, se muestre o no el detalle de la fila. Es una decision consciente: el desplegable de usuarios sale de ``sellers``, entonces hace falta traerlo siempre. El ``sales`` es el candidato natural a recortarse algun dia molesta.

6. Las cinco reglas que quedaron

REGLA	DE DONDE SALE
Un <code>*ngIf</code> que envuelve el control tambien borra el control	Tabla que desaparecia
Un menu no puede armarse con una lista ya recortada	Usuarios que desaparecian
Si la tabla ya tiene los datos, el total es una suma	Corte de caja
Un dato que no corresponde se muestra como ausente, no como cero	Diferencia con filtro
Si dos componentes muestran el mismo documento, el CSS va en el global	Formato del corte

Y la causa comun, que es la que mas conviene retener: las tres fallas estaban en la CAPA que no se estaba mirando. Un ``nu`` de mas en el frontend borro una seccion entera; un ``continue`` razonable en el backend vacio un desplegable; y un endpoint correcto usado donde no tocaba mostro el turno equivocado. Ninguno de los tres se arregla mirando la pantalla.

LA PREGUNTA QUE LAS TRES COMPARTEN

No "como lo arreglo" sino "que dato estoy mirando". En los tres casos, la respuesta fue el nombre de un campo: ``cashBoxReport``, ``sellers``, ``getSummary()``. El sintoma siempre esta dos saltos mas alla del archivo que se estaba editando.